App Dev 1 Project Report

1. Student Details

Name: Disha Bhandari
Roll Number: 24F2002244
Email: 24f2002244@ds.study.iitm.ac.in

About Me:
I am a student at IIT Madras BS Degree program in Data Science and Applications (diploma level) with a strong interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

---

2. Project Details

Project Title: Placement Portal Application

Problem Statement:
To design and build a web-based placement portal that enables administrators, companies, and students to efficiently manage and participate in the campus recruitment process. The system allows companies to post job drives, students to apply for opportunities, and administrators to oversee and control the entire workflow.

Approach:
The application was developed using Flask as the backend framework with a modular structure. It facilitates:

- Company registration and job drive creation
- Student profile management and job applications
- Admin control for approving companies and managing drives
- Application tracking with status updates (shortlisted, rejected, waiting)
- Prevention of duplicate job applications for the same drive
- Basic data visualization using charts for better insights
- Secure authentication with role-based access control using Flask-Login

The system ensures a structured and streamlined placement process by integrating authentication, role-based access, real-time updates, and organized data management.

---

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist in writing SQLAlchemy model definitions, improving variable naming consistency, and getting guidance for integrating Flask-Login for authentication.

The extent of AI/LLM usage is around 10–15%, limited to code suggestions, minor implementation guidance, and documentation formatting.

All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualize placement trends and application statistics
Flask-Login	User authentication, session management, and route protection
HTML5	Frontend and form validation
SQLite	Lightweight local database for storing application data

---

5. Database Schema / ER Diagram

Tables:
Admin — stores administrator details
(id, name, email, password)

Student — stores student profile information
(id, name, email, password, department, resume_link, is_blacklisted)

Company — stores company details
(id, name, email, password, location, is_approved, is_blacklisted)

Drive — stores job postings created by companies
(id, company_id, job_title, job_description, eligibility_criteria, application_deadline, salary, status, is_rejected, is_completed, completed_on)

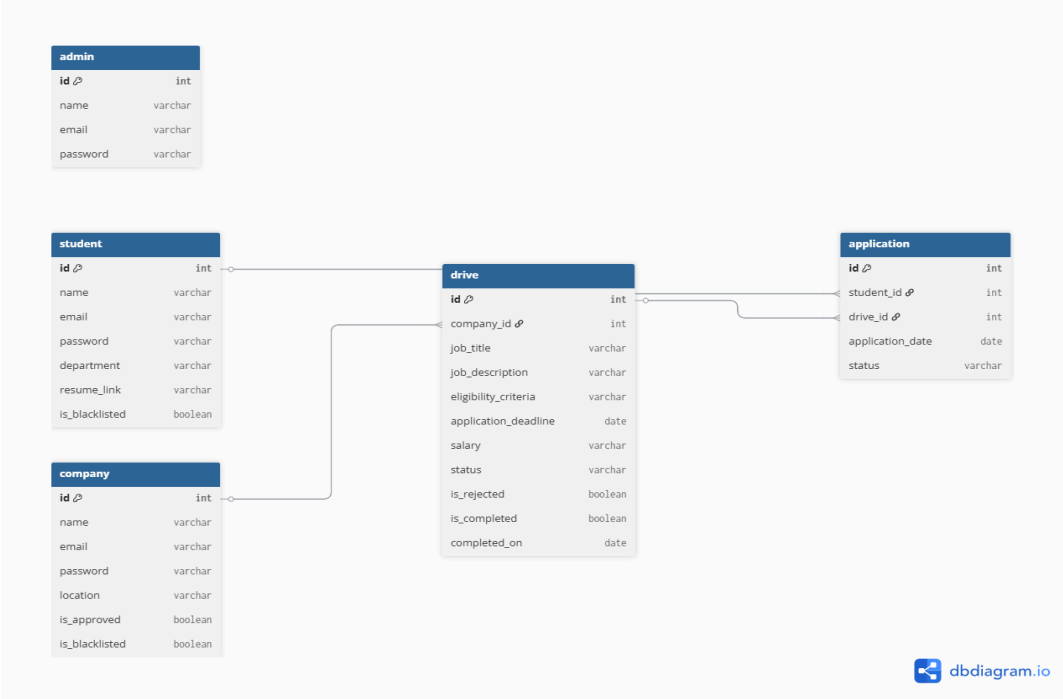
Application — stores student applications for drives
(id, student_id, drive_id, application_date, status)

Relationships:
One-to-Many → Company → Drive
One company can create multiple job drives

One-to-Many → Student → Application
One student can apply to multiple drives

One-to-Many → Drive → Application
One drive can have multiple student applications

Many-to-Many → Student ↔ Drive (via Application)
Students can apply to multiple drives, and each drive can have multiple students



---

6. Architecture and Features

Architecture Overview:
app.py – main Flask application entry point handling routes and app configuration

models/ – database models using SQLAlchemy (Admin, Student, Company, Drive, Application)
controllers / routes – Flask routes handling logic for admin, student, and company operations
templates/ – Jinja2 HTML templates for rendering dashboards and pages
static/ – CSS, Bootstrap, and JavaScript files for UI styling and interactivity

---

Implemented Features

User Authentication System

- Separate login for Admin, Student, and Company
- Secure session handling using Flask-Login
- Role-based user loading using custom user_loader
- Protected routes using `@login_required`
- Role-based and ID-based access control to prevent unauthorized access

---

Admin Dashboard

- Approve or blacklist companies
- Manage students and job drives
- Monitor all applications
- View summary statistics using charts

Company Dashboard

- Create and manage job drives
- View received applications
- Shortlist, reject, or update application status
- View application trends for posted drives

Student Dashboard

- View approved companies and job postings
- Search job postings by company name and job title
- Apply to job drives with duplicate application prevention
- Track application status (Applied, Shortlisted, Rejected, Waiting)
- View application status distribution using charts

Application Management System

- Real-time status updates
- Centralized tracking of all applications

Form Validation

- Frontend validation using HTML5 and JavaScript
- Backend validation in Flask routes

Responsive UI

- Built using Bootstrap for clean and mobile-friendly design

Additional Features

Application History
Students can view past applications and their outcomes

Drive Status Management
Drives can be marked as completed or rejected

Resume Integration
Students can upload and share resume links with companies

Role-Based Access Control

- Admin, Company, and Student have different permissions
- Unauthorized access to routes is restricted using Flask-Login

---

7. Video Presentation

Drive Link:
https://drive.google.com/file/d/1rtRS6d47JQBT0pP-iDpbHthWplm2COs_/view?usp=sharing